

Phase 1 Documentation

Student Number: s5331680

Name: Hadi Alkhub

Workshop Time: Wednesday 3pm

GitHub Repository: <https://github.com/HadiAlk4/Chat-App-System>

1 Project Overview

This project is a full-stack chat application built using the MEAN stack (Angular, Node.js, Express, with MongoDB and Socket.io deferred to Phase 2). The system facilitates single-channel, single-thread communication organized by Groups and sub-Channels (Rooms). The frontend utilizes PaperCSS for a responsive, less-formal UI design. The application features a strict three-tier permission system (Super Admin, Group Admin, User) to manage content, memberships, and administrative requests.

2 Git Strategy

The version control strategy relies on feature-based branching to keep the codebase stable:

- **Frontend:** Core UI development and Angular component routing are tracked directly on the `main` branch.
- **Backend:** Server setup, Express routing, and file persistence are isolated on a dedicated `phase1/backend` branch.
- **Review & Merge:** Once backend routes are successfully tested against the frontend, `phase1/backend` is merged into `main`. A final `chore/cleanup` branch is utilized to review the entire integrated system, remove dead code, and ensure formatting before the final Phase 1 submission.

3 Specifications & Assumptions

The following table outlines the functional requirements, system specifications, and our assumptions for the Phase 1 prototype. Because Phase 1 focuses on UI design and user management, complex features like live sockets are mocked using hardcoded component data.

Category	Specification / Assumption for Phase 1 Prototype
Tech Stack	MEAN stack methodology is used. However, as per Phase 1 guidelines, MongoDB and Socket.io are deferred to Phase 2. Local JSON file storage (<code>fakeData.json</code>) is used for backend persistence.
UI & Styling	The application interface is built using Angular and styled with the PaperCSS framework to meet the "less formal" and responsive design requirements.
User Authentication	Basic authentication is functional. The Node.js backend verifies email and password combinations against <code>fakeData.json</code> and returns the user object to be stored locally in the browser's <code>sessionStorage</code> .
Roles & Permissions	A three-tier permission system (Super Admin, Group Admin, User) is written in a way to that is planned to be implemented in Phase two based on their <code>sessionStorage</code> role.
Group & Room Creation	The UI for proposing groups and creating rooms is fully built using PaperCSS modals and forms. For Phase 1, this data is mocked and hardcoded locally using arrays and objects directly within the Angular <code>.ts</code> files.
Chat & Messaging (Mocked)	Chat messages are currently hardcoded as mock arrays inside <code>chat.ts</code> to demonstrate the UI layout.
Admin Queues (Mocked)	Complex approval queues (group deletion, ban requests) and the Super Admin Audit Log are built into the UI but rely on mocked local component data arrays rather than backend endpoints for Phase 1.
Account Deletion	The UI provides a "Request Account Deletion" button. The deletion of user messages via WebSockets will be implemented in Phase 2.

4 Data Structures

For Phase 1, server-side data is persisted in a local `fakeData.json` file.

- **users Array:** Stores user objects containing `id`, `username`, `email`, `password`, `birthdate`, `age`, `role`, and a `valid` boolean flag.
- **groups Array:** Stores group objects containing `id`, `name`, `description`, `minAge`, `themeColor`, and string arrays mapping `admins` and `members`.
- **rooms Array:** Stores individual chat channels containing an `id`, `name`, and a `groupId` acting as a relational link to its parent group.

Note: For Phase 1, auxiliary data structures like Join Requests, Audit Logs, and Chat Messages are mocked as arrays directly within their respective Angular `.ts` component files.

5 Proposed Angular Architecture

The architecture splits responsibilities between the Node/Express backend and the Angular frontend.

- **Server-Side Responsibility:** The backend strictly handles User Authentication (Login) and User Registration (Signup) alongside basic Group/Room array appending.
- **Client-Side Responsibility:** All other application states (chat history, pending requests, audit logs, member tables) are managed locally using mocked datasets inside the `.ts` files for each screen. Data transfer relies on Angular's two-way data binding (`[(ngModel)]`).

Core Components & Routes:

- `/login` (**LoginComponent**): Entry point, validates credentials via HTTP POST.
- `/signup` (**SignupComponent**): Registers new users via HTTP POST.
- `/dashboard` (**DashboardComponent**): Displays available groups and handles group proposals via PaperCSS modals.
- `/my-memberships` (**MyMembershipsComponent**): Lists joined groups.
- `/group-settings` (**GroupSettingsComponent**): Group Admin dashboard to manage rooms and members.
- `/chat` (**ChatComponent**): Main chat interface.
- `/super-admin-dashboard` (**SuperAdminDashboardComponent**): Global request approvals.
- `/super-admin-audit-log` (**SuperAdminAuditLogComponent**): Global action ledger.

6 Proposed Server Endpoints

The Phase 1 Express API uses the `fs` module to read and write to `fakeData.json`.

- **POST** `/api/auth`: Accepts `{ email, password }`. Returns `{ ok: true, user }` if credentials match.
- **POST** `/api/signup`: Accepts new user data, appends it to the `users` array, and writes to the JSON file.
- **GET** `/api/groups`: Returns the current array of active groups. (Note: Currently mocked in the frontend `.ts` files for Phase 1.)

- **POST /api/groups:** Appends a new group object to the `groups` array. (Note: Currently mocked in the frontend `.ts` files for Phase 1.)
- **POST /api/rooms:** Finds a specific group and appends a new room name to its `rooms` array. (Note: Currently mocked in the frontend `.ts` files for Phase 1.)

7 Design Documents & Storyboards

UI mockups and responsive design storyboards for desktop and tablet screens have been completed and are stored within the `assignment_material` folder of this repository.